

TRACK 2 | AI AGENT APPLICATION

RDK Agent

A safety-governed multi-agent system that turns natural-language intent into verified actions on RDK development boards.

PRIVATE AMD RADEON INFERENCE / TOOL CALLING / MULTI-STEP PLANNING / PERMISSION BOUNDARIES

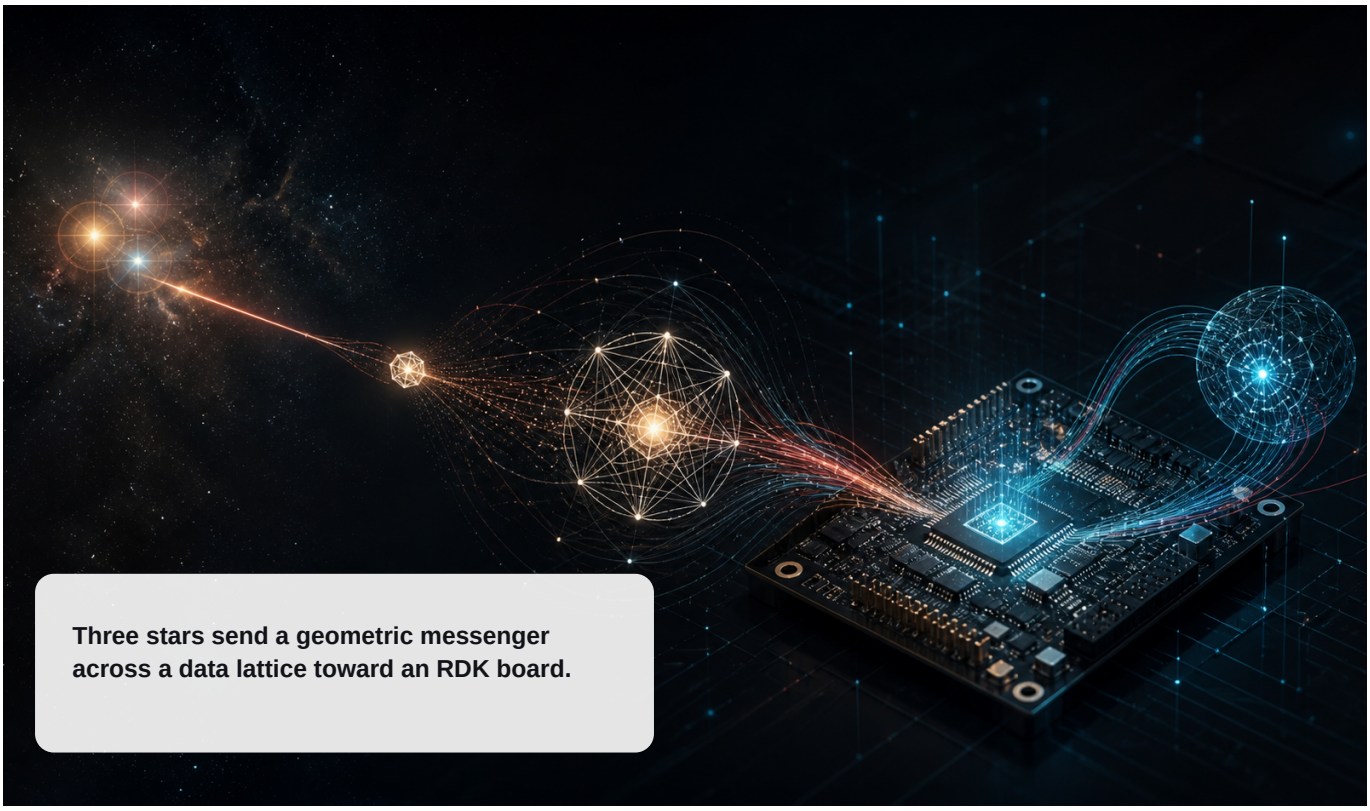
Team / participant: d-robotics agent

Submission build: 2026-08-05

NAMING STORY

Why 'Sophon'? A literary metaphor made operational

The name connects a memorable science-fiction idea to a transparent, owner-controlled device agent.



Three stars send a geometric messenger across a data lattice toward an RDK board.

FICTIONAL METAPHOR

A Sophon is sent toward Earth to observe and communicate.

RDK-SOPHON

A board-side probe observes telemetry, health and available actions.

RDK-AGENT

The host-side agent reasons, calls tools and requests governed board operations.

The crucial difference

The fictional Sophon is covert. RDK Sophon is deliberately transparent: deployed by the board owner, constrained by explicit action packages, logged, auditable and reversible. The metaphor describes proximity and communication - not surveillance without consent.

Project-created AI-generated concept illustration; no official artwork or adaptation assets are used. This independent project is not endorsed by or affiliated with the work's author, publishers, rights holders, or screen adaptations.

PRODUCT VALUE

One intent, one governed path to a real board

RDK Agent removes the fragile handoffs between coding, verification, packaging, deployment and device acceptance.

Developer intent

"Add or repair a board capability, prove it, then deploy it safely."

01 Build with evidence

A coding agent works inside explicit write paths; tests and verification artifacts are required before the action package can advance.

02 Deploy atomically

The release step validates a deterministic package, deploys it to the board and keeps rollback information.

03 Operate in natural language

The installed Skill exposes board actions to the agent while the board daemon enforces the device-side boundary.

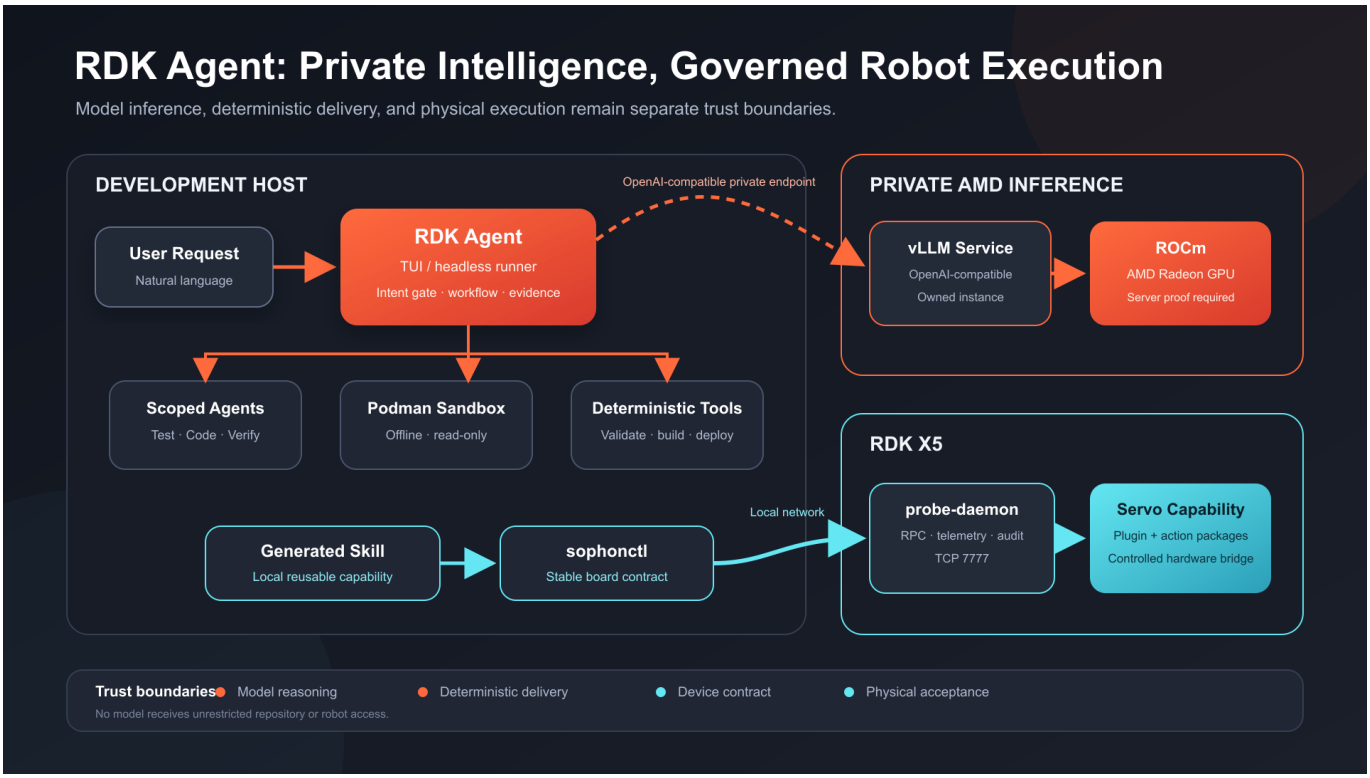
Definition of done

- The requested action exists, passes executable checks and respects direction policy.
- The board responds after deployment, and acceptance can be repeated from CLI or agent chat.

ARCHITECTURE

Reasoning stays private; device authority stays local

The host orchestrates; AMD Radeon inference supplies the model; rdk-sophon owns the final board-side action boundary.



HOST

Pi SDK/TUI coordinates scoped agents, Skills, tools and evidence.

AMD RADEON

An OpenAI-compatible vLLM endpoint provides private model inference.

RDK X5

probe-daemon exposes telemetry and allowlisted plugins over local RPC.

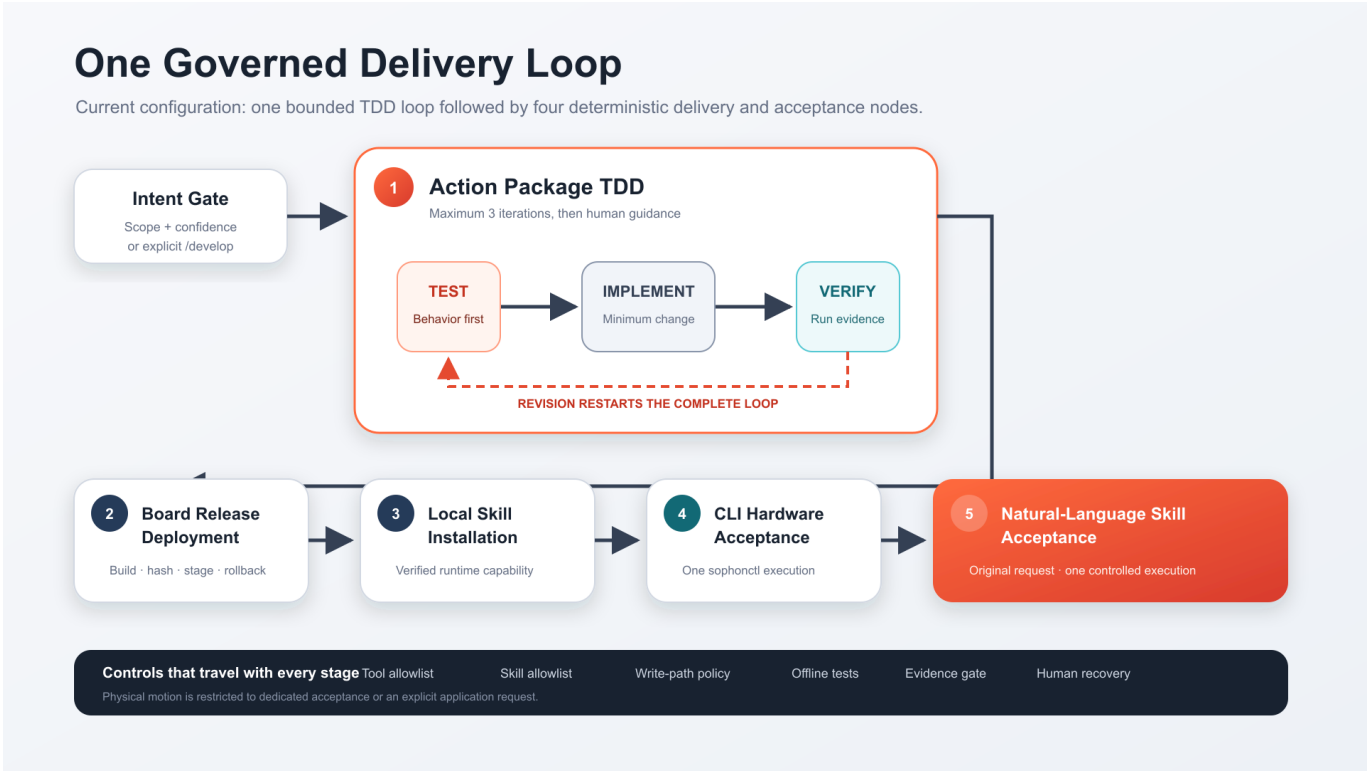
Trust boundary

No prompt can directly bypass action-package validation, executable evidence gates or the daemon's plugin boundary.

AGENT WORKFLOW

Five nodes turn a request into a verified board action

The implementation uses one Action Package TDD loop (maximum three iterations), then release, Skill installation and two acceptance checks.



1 Test | 2 Coding | 3 Verification | 4 Board release | 5 Skill + acceptance

Guardrails remain active at every transition: scoped tools, scoped writes, read-only sandboxing, direction policy and executable evidence.

CORE CAPABILITIES

Three Track 2 capabilities, backed by evidence

The capability claims below are backed by repository implementation, tests and board-side evidence.

01

Tool calling

Agents use allowlisted tools and Skills to inspect code, run checks, package actions, deploy and query the board.

02

Multi-step planning

A five-node graph coordinates TDD, release, installation and acceptance with bounded retry behavior.

03

Agent/tool permissions

Role scopes, write-path scopes, offline read-only sandboxes and the device daemon constrain authority. Transport authentication and routine per-action approval remain roadmap items.

Honest boundary

The current system has in-process conversation context, but not a persistent local memory store. We do not claim RAG or persistent multi-turn memory in this submission.

SAFETY MODEL

The control plane is stronger than the prompt

RDK Agent assumes models can be wrong; permissions, policy and evidence are enforced by software below the model layer.

SCOPE	**Agent-specific tool, Skill and write-path allowlists**
SANDBOX	**Offline, read-only Podman execution for untrusted verification**
DIRECTION	**ACTION-DIRECTION-001 blocks commands that violate action direction**
EVIDENCE	**Executable checks must pass before a package can progress**
DEVICE	**probe-daemon exposes only registered board plugins and actions**

Result

The model can propose. It cannot silently expand its own authority, bypass evidence, or directly mutate the board outside the daemon contract.

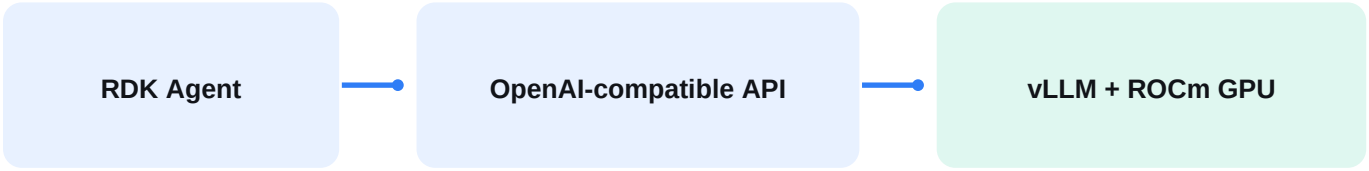
Two Qwen3 model paths ran on Radeon

The 80B model completed a single-GPU serving run; the trained 32B SFT model also completed the live five-node RDK Agent workflow.

BOTH MODELS RUN

Qwen3-Next-80B-A3B-Instruct

80B: single-Radeon llama.cpp serving + API canaries | 32B SFT: five-node live Agent run, 5/5 in 4 min 04 s



Implemented inference reductions

- Role-specific prompts and scoped context reduce irrelevant tokens.
- Action packages and structured handoffs reduce free-form retry loops.
- Bounded TDD iterations cap runaway agent work.

Archived model evidence

- Both paths identify the gfx1100 Radeon GPU and model weights.
- Baseline-versus-optimized metrics are archived for both paths.
- Only the separate private vLLM server provenance remains unarchived.

MEASURED PERFORMANCE

Two Radeon paths, two clearly scoped evidence sets

The repository archives measured baseline-versus-optimized results for the 32B SFT and 80B serving paths without conflating their runtimes.

TTFT

Time to first token

Measures interactive responsiveness

TOTAL

End-to-end latency

Measures user-perceived task delay

TOK/S

Output throughput

Measures serving efficiency

Benchmark sequence

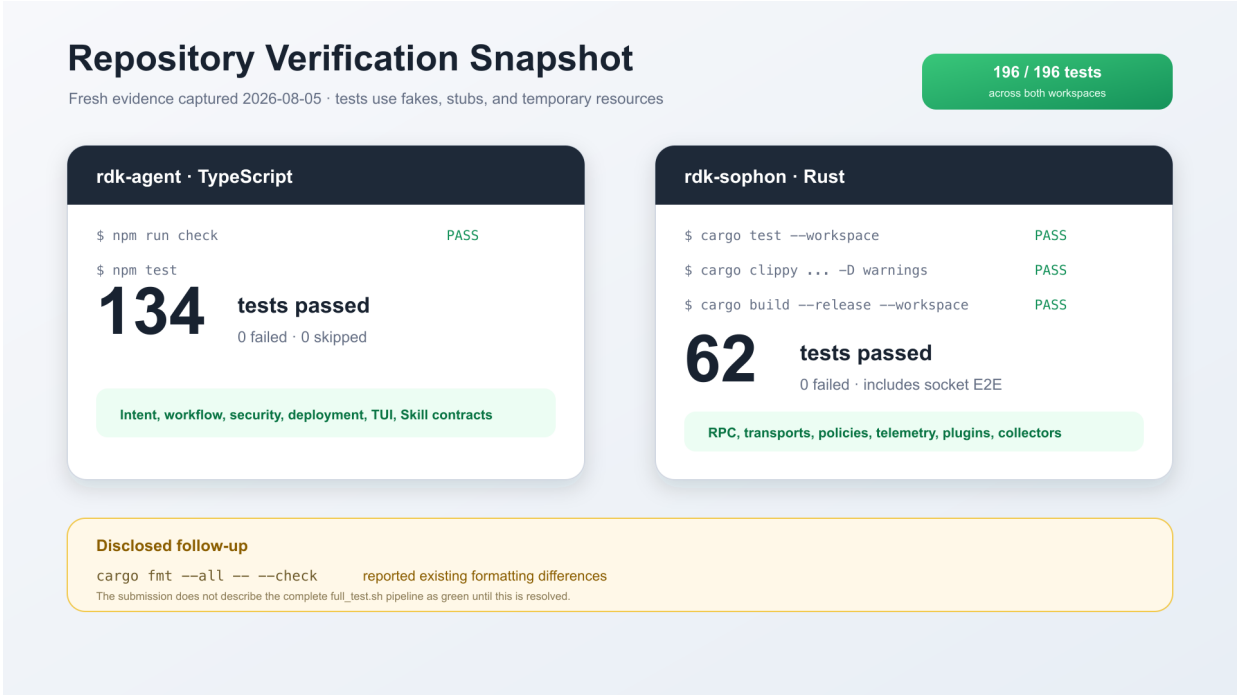
01	**32B SFT**	88 trials per arm; TTFT p50 improved from 17.41 s to 8.26 s.
02	**80B**	Single Radeon; decode improved from 37.19 to 49.82 tok/s.
03	**Agent**	The 32B SFT model completed all five live workflow nodes in 4 min 04 s.
04	**Audit**	Raw or aggregate records, hashes and verification scripts remain in the repository.

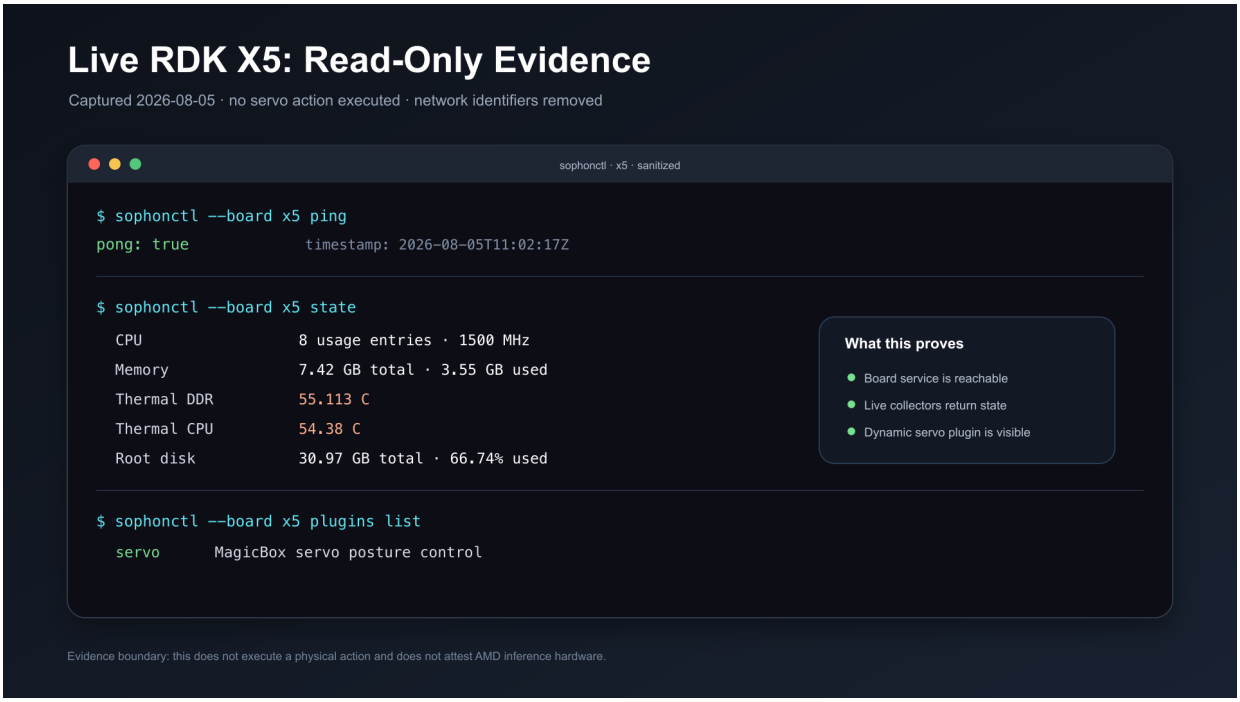
Evidence boundary: the private vLLM endpoint is verified only at the client-routing level; the 80B numbers above come from the separately archived llama.cpp run.

VERIFIED EVIDENCE

196 tests pass, and a live RDK X5 answers

Evidence below was captured on 2026-08-05. Sensitive identifiers were intentionally removed.





TypeScript: 134/134 | Rust: 62/62 | clippy -D warnings: pass | release build: pass

Read-only board checks: ping, state, plugins list

REPRODUCIBILITY

The demo can be repeated from source to board

The source repository includes environment setup, startup commands, dependency notes and a sanitized AMD model configuration example.

HOST	BOARD	DEMO
01 Node.js 22+	01 Rust toolchain	01 Start probe-daemon
02 npm install	02 cargo test --workspace	02 Run rdk-agent
03 npm run check	03 cargo clippy --workspace -- -D warnings	03 Issue natural-language task
04 npm test	04 cargo build --release --workspace	04 Show action evidence
05 npm run build		05 Verify board result

Demo evidence to show on screen

- Natural-language request in the TUI, then visible graph progress.
- Tests and action-package validation before deployment.
- Board ping/state/plugin acceptance after deployment.
- AMD Radeon / ROCm server evidence with secrets redacted.

SUBMISSION READINESS

The package is assembled and evidence-backed

Team identity, public demo links, both Qwen3 model runs and measured Radeon results are now recorded.

DONE	**Project specification, architecture and workflow**
DONE	**Source README, dependencies and reproducibility guide**
DONE	**PPT pitch deck and original visual assets**
DONE	**196-test and live-board evidence**
DONE	**Team: d-robotics agent**
DONE	**Public demo link: Bilibili; Baidu public share URL pending**
DONE	**32B SFT and 80B Radeon run evidence**

Demo video

Bilibili: BV1t3up6iEhy

Backup: Baidu public share URL pending

Primary references

Official repository: github.com/AMD-DEV-CONTEST/Radeon-hackathon-2026-07
Event page: luma.com/amd-4dhi
Project evidence: submission/en/evidence/verification-2026-08-05.md